Studio Nx

JS 01 — refreshAccessToken()

2026-10-12

JS 01 — `refreshAccessToken()`

항목	내용
파일	`studionx/frontend/static/js/core/api.js`
작업	P1-02 프론트 공통 JS (PRD 01 FR-105)
작성	Claude (AI), 2026-10-06, 사용자 검토
다음	02 `apiFetch()`

1. 하는 일

리프레시 쿠키로 Access Token을 다시 받습니다. 성공하면 `true`, 실패하면 `false`를 돌려줍니다.

2. 언제 쓰나

Access Token은 메모리에만 있어서 **페이지를 옮기면 사라지고**, **30분이 지나면 만료**됩니다. 그때 로그인을 다시 하지 않고 토큰만 새로 받으려고 이 함수를 씁니다.

상황	부르는 곳
페이지를 열고 첫 API 요청을 보낼 때 (토큰이 아직 없음)	`apiFetch()`가 요청 전에 먼저 부릅니다
요청했는데 401이 왔을 때 (토큰 만료)	`apiFetch()`가 재발급한 뒤 같은 요청을 한 번 더 보냅니다
로그인이 필요한 페이지에 들어올 때	`isLoggedIn()` → `requireLogin()` (마이페이지, 내 프로젝트, 회원 정보, 시퀀서)
헤더를 로그인 상태로 바꿀 때	`isLoggedIn()` (모든 페이지의 헤더)

페이지 코드가 이 함수를 직접 부르는 일은 거의 없고, 위의 함수들을 거쳐서 부릅니다.

리프레시 쿠키 (`snx_refresh`)

로그인을 오래 유지하려고 서버가 브라우저에 심는 쿠키이며, 안에 리프레시 토큰이 들어 있습니다. (서버 쪽은 P1-05의 `api/v1/auth.py` 입니다.)

언제	무슨 일
로그인할 때 (`POST /auth/login`)	서버가 쿠키를 심습니다. **자동 로그인을 체크**하면 3일 동안 유지되고(브라우저를 닫아도 남음), **체크하지 않으면** 세션 쿠키가 됩니다(브라우저를 닫으면 사라짐)
재발급할 때 (`POST /auth/refresh`, 이 함수)	브라우저가 쿠키를 자동으로 보내고, 서버가 확인한 뒤 새 Access Token을 줍니다
로그아웃할 때 (`POST /auth/logout`)	서버가 쿠키를 지웁니다
그 밖의 API 요청	쿠키가 **실리지 않습니다**. `path=/api/v1/auth`로 범위를 좁혀 두었기 때문입니다

속성	뜻
`HttpOnly`	JS에서 읽을 수 없습니다. 그래서 스크립트 공격(XSS)으로 훔칠 수 없습니다

속성	뜻
`SameSite=Lax`	다른 사이트가 보낸 POST에는 실리지 않습니다. CSRF에 대비한 설정입니다
`Secure` (배포)	HTTPS에서만 전송합니다
`path=/api/v1/auth`	로그인, 재발급, 로그아웃 요청에만 실립니다

토큰	저장 위치	수명	쓰임
Access Token	메모리 (`api.js`의 변수)	30분	모든 API 요청의 `Authorization: Bearer …`
리프레시 토큰	HttpOnly 쿠키	3일 또는 브라우저를 닫을 때까지	Access Token을 다시 받을 때만 씁니다

쿠키가 없거나 만료되면 `/auth/refresh`가 401로 응답하고, 이 함수는 `false`를 돌려주므로 로그인하지 않은 상태로 처리됩니다.

3. 코드

import (파일 맨 위) `api.js`는 다른 파일을 import하지 않습니다 (공통 모듈의 맨 아래 단계입니다). 이 함수가 쓰는 `refreshing`, `setAccessToken`, `clearAccessToken`, `API_BASE`는 같은 파일 안에 있고, `fetch`는 브라우저의 기본 기능입니다.

함수

```
export async function refreshAccessToken() {
  if (refreshing) return refreshing;
  refreshing = (async () => {
    try {
      const res = await fetch(`${API_BASE}/auth/refresh`, {
        method: 'POST',
        credentials: 'same-origin',
        headers: { Accept: 'application/json' },
      });
      const data = res.ok ? await res.json().catch(() => null) : null;
      if (!data?.access_token) {
        clearAccessToken();
        return false;
      }
      setAccessToken(data.access_token);
      return true;
    } catch {
      clearAccessToken();
      return false;
    } finally {
      refreshing = null;
    }
  })();
  return refreshing;
}
```

문법 풀이

import 문법	뜻
`export` (함수 앞)	다른 파일이 `import { 함수 }`로 가져다 쓸 수 있게 내보냅니다. 없으면 이 파일 안에서만 쓸 수 있습니다

`if (!data?.access_token)`: "토큰을 받지 못했으면"이라는 뜻입니다.

부분	뜻
`data?.access_token`	**옵셔널 체이닝(`?.`)**입니다. `data`가 `null`이나 `undefined`이면 오류 없이 `undefined`가 되고, 아니면 `data.access_token`이 됩니다
`!`	값이 없거나(`undefined`, `null`) 빈 문자열이면 `true`가 됩니다

`data`	`data?.access_token`	`!data?.access_token`	결과
`null` (실패 응답이거나 JSON이 아님)	`undefined`	`true`	실패로 처리합니다
`{}` (토큰 없음)	`undefined`	`true`	실패로 처리합니다
`{ access_token: '' }`	`''`	`true`	실패로 처리합니다
`{ access_token: 'eyJ…' }`	`'eyJ…'`	`false`	성공으로 처리합니다

`?.` 없이 `data.access_token`이라고 쓰면, `data`가 `null`일 때 `TypeError: Cannot read properties of null`로 멈춥니다. 같은 뜻을 길게 쓰면 `if (data === null || data === undefined || !data.access_token)`입니다.

`res.ok ? await res.json().catch(() => null) : null`: 성공 응답(200~299)일 때만 본문을 JSON으로 읽고, 읽다가 실패하면 `null`이 됩니다. 실패 응답이면 읽지 않고 `null`이 됩니다. 그래서 아래의 `if` 하나로 모든 실패를 거를 수 있습니다.

4. 흐름

1. 재발급이 이미 진행 중이면 그 Promise를 돌려줍니다.
2. `POST /api/v1/auth/refresh`를 보냅니다. 쿠키는 브라우저가 자동으로 싣습니다.
3. 응답에 `access_token`이 있으면 저장하고 `true`를 돌려줍니다.
4. 401, 네트워크 오류, 토큰이 없는 응답이면 토큰을 지우고 `false`를 돌려줍니다.
5. 끝나면 `refreshing`을 비웁니다.

5. 왜 이렇게

선택	이유
진행 중인 Promise를 공유(`refreshing`)	여러 요청이 동시에 401을 받아도 재발급은 한 번만 합니다. 리프레시 토큰을 한 번 쓰고 바꾸는 방식에서도 안전합니다
토큰을 JS가 넣지 않음	리프레시 토큰은 HttpOnly 쿠키라서 JS에서 보이지 않습니다(XSS 대비). `credentials: 'same-origin'`이면 자동으로 전송됩니다
실패해도 예외를 던지지 않음	부르는 쪽(`apiFetch`, `isLoggedIn`)은 성공 여부만 알면 됩니다
`res.json().catch(() => null)`	JSON이 아닌 응답도 예외 없이 실패로 처리합니다
`finally`에서 비움	성공이든 실패든 상관없이, 다음 호출은 새로 요청하게 합니다

6. 시험 (fetch를 가짜로 바꿔서 시험: `/auth/refresh`는 P1-05에서 구현)

경우	결과
200 + `access_token`	`true`를 돌려주고 토큰을 저장합니다. 주소, POST, 쿠키 옵션도 확인했습니다

경우	결과
401	`false`를 돌려주고 토큰을 지웁니다
네트워크 오류	`false`를 돌려주고 토큰을 지우며, 예외는 나지 않습니다
200인데 토큰이 없음	`false`
동시에 3번 호출	fetch는 1번만 하고, 셋 다 `true`입니다
끝난 뒤 다시 호출	새로 요청해서 새 토큰을 받습니다